

Optimal Mixing of Passive Scalars

Expanded Mathematical, Algorithmic, and Computational Guide

Adebanji Adelowo

September 2026

Abstract

This companion report expands the mathematical and computational ideas in the thesis *Optimal Mixing of Passive Scalars*. It derives the Lin–Thiffeault–Doering velocity step by step, explains the Fourier pseudo-spectral discretisation from continuous operators to NumPy arrays, and connects each equation to the corresponding Python implementation. It also describes the adaptive Runge–Kutta solver, the conservation-based stopping event, the extraction of mixing rates, and the interpretation of the resolution study. The aim is not to replace the original thesis, but to make its reasoning and code easier to study, reproduce, and extend.

Contents

1	Problem in Plain Language	3
2	Why the H^{-1} Norm Measures Mixing	3
3	Deriving the Instantaneously Optimal Velocity	4
3.1	Step 1: Differentiate the objective	4
3.2	Step 2: Keep only the divergence-free component	4
3.3	Step 3: Express the objective in the enstrophy inner product	4
3.4	Step 4: Saturate the stirring budget	4
3.5	The degenerate case	5
4	From Fourier Analysis to NumPy Arrays	5
4.1	Grid and transform convention	5
4.2	Wavenumbers and derivatives	5
4.3	Inverse Laplacian and zero mode	6
4.4	Discrete norm factors	6
5	The Right-Hand Side Algorithm	6
5.1	Why products are formed in physical space	7
6	State Representation and Adaptive Time Integration	7
7	Resolution Check as an Event Function	8
8	Initial Data and Normalization	8

9	Extracting and Interpreting Mixing Rates	9
10	End-to-End Code Map	9
11	Numerical Risks and Recommended Extensions	10
12	Summary	10

1 Problem in Plain Language

A passive scalar is a quantity carried by a flow without changing that flow. Dye in water is the standard example. The scalar field $\theta(x, y, t)$ records the dye concentration, while $u(x, y, t)$ is the velocity of the fluid. With no molecular diffusion, the governing equation is

$$\partial_t \theta + u \cdot \nabla \theta = 0, \quad \nabla \cdot u = 0. \quad (1)$$

The first relation says that the concentration observed along a fluid trajectory does not change. The second says that the flow is incompressible: it rearranges area but does not create or destroy it.

Mixing therefore does not mean that θ becomes pointwise smaller. Instead, stirring stretches large patches into increasingly thin positive and negative filaments. A coarse observer sees those nearby values cancel, even though the full distribution of scalar values is preserved.

The computational question is:

Given a fixed stirring budget, which incompressible velocity produces the largest instantaneous decrease in a measure of unmixed large-scale structure?

The LTD strategy answers this question at every instant. It is a greedy strategy: locally optimal in time, but not necessarily optimal over an entire finite horizon [1].

2 Why the H^{-1} Norm Measures Mixing

Let the periodic domain be $\mathbb{T}^2 = [0, 1]^2$, and write a mean-zero scalar as

$$\theta(x) = \sum_{k \in \mathbb{Z}^2 \setminus \{0\}} \hat{\theta}_k e^{2\pi i k \cdot x}. \quad (2)$$

Its squared L^2 norm is

$$\|\theta\|_{L^2}^2 = \sum_{k \neq 0} |\hat{\theta}_k|^2. \quad (3)$$

Every Fourier mode receives the same weight. In contrast,

$$\|\theta\|_{H^{-1}}^2 = \sum_{k \neq 0} \frac{|\hat{\theta}_k|^2}{4\pi^2 |k|^2}. \quad (4)$$

The factor $|k|^{-2}$ strongly weights low wavenumbers and suppresses high wavenumbers. Transferring scalar variance from a broad structure ($|k|$ small) to a fine filament ($|k|$ large) therefore reduces the H^{-1} norm without changing the L^2 norm.

Worked example 2.1. Suppose one unit of Fourier energy lies entirely in a mode with $|k| = 1$. Its contribution to (4) is $1/(4\pi^2)$. Moving the same energy to $|k| = 10$ leaves the L^2 contribution equal to one, but reduces the squared H^{-1} contribution by a factor of 100. This is precisely the large-to-small-scale transfer that mixing is meant to detect.

If $\phi = \Delta^{-1} \theta$, so that $\Delta \phi = \theta$, integration by parts on the torus gives an equivalent physical-space formula:

$$\|\theta\|_{H^{-1}}^2 = - \int_{\mathbb{T}^2} \theta \phi \, dx \, dy = \int_{\mathbb{T}^2} |\nabla \phi|^2 \, dx \, dy. \quad (5)$$

This identity is central to the velocity derivation.

3 Deriving the Instantaneously Optimal Velocity

3.1 Step 1: Differentiate the objective

Differentiate (5). Since Δ^{-1} is self-adjoint, the two differentiated terms are equal:

$$\frac{d}{dt} \|\theta\|_{H^{-1}}^2 = -2 \int_{\mathbb{T}^2} (\partial_t \theta) \Delta^{-1} \theta \, dx \, dy \quad (6)$$

$$= 2 \int_{\mathbb{T}^2} (u \cdot \nabla \theta) \phi \, dx \, dy, \quad (7)$$

where (1) was used in the second line. Incompressibility and periodicity move the derivative from θ to ϕ :

$$\frac{d}{dt} \|\theta\|_{H^{-1}}^2 = -2 \int_{\mathbb{T}^2} u \cdot \underbrace{(\theta \nabla \Delta^{-1} \theta)}_g \, dx \, dy. \quad (8)$$

3.2 Step 2: Keep only the divergence-free component

The vector field g need not be divergence-free. The Leray projection

$$Pg = g - \nabla \Delta^{-1} (\nabla \cdot g) \quad (9)$$

removes its gradient component. For every divergence-free u , $\int u \cdot \nabla q \, dx \, dy = 0$, so

$$\int u \cdot g \, dx \, dy = \int u \cdot Pg \, dx \, dy. \quad (10)$$

The discarded component cannot affect the objective under the admissible velocity constraint.

3.3 Step 3: Express the objective in the enstrophy inner product

Define

$$v = -\Delta^{-1} Pg. \quad (11)$$

Then $Pg = -\Delta v$, and another integration by parts yields

$$\frac{d}{dt} \|\theta\|_{H^{-1}}^2 = -2 \int_{\mathbb{T}^2} \nabla u : \nabla v \, dx \, dy. \quad (12)$$

The colon denotes the Frobenius product: sum the products of all four entries of the two velocity-gradient matrices.

3.4 Step 4: Saturate the stirring budget

The admissible velocities satisfy $\|\nabla u\|_{L^2} \leq F$. By Cauchy–Schwarz,

$$\int \nabla u : \nabla v \, dx \, dy \leq \|\nabla u\|_{L^2} \|\nabla v\|_{L^2} \leq F \|\nabla v\|_{L^2}. \quad (13)$$

Equality holds when u points in the same direction as v in the gradient inner product and uses the entire budget. Hence

$$\boxed{u = F \frac{v}{\|\nabla v\|_{L^2}}, \quad v = -\Delta^{-1} P(\theta \nabla \Delta^{-1} \theta).} \quad (14)$$

Substitution into (12) gives

$$\frac{d}{dt} \|\theta\|_{H^{-1}}^2 = -2F \|\nabla v\|_{L^2} \leq 0. \quad (15)$$

Thus the method cannot increase the mix norm unless numerical error overwhelms the continuous argument.

3.5 The degenerate case

If $Pg = 0$, then $v = 0$ and the normalization in (14) is undefined. Mathematically, every admissible velocity has zero first-order effect on the mix norm. The LTD paper treats this case through a second-derivative optimization and an eigenvalue problem [1]. The implementation does not solve that secondary problem; it compares $\|\nabla v\|_{L^2}$ with $\|\Delta^{-1}g\|_{L^2}$ and prints a warning when their ratio becomes very small.

4 From Fourier Analysis to NumPy Arrays

4.1 Grid and transform convention

The code samples the unit torus on an $N \times N$ grid with spacing $h = 1/N$. NumPy's forward transform is unnormalized and its inverse includes $1/N^2$. With the array indexing convention used in the code,

$$\hat{f}[k_y, k_x] = \sum_{i,j=0}^{N-1} f[i, j] e^{-2\pi i(i k_y + j k_x)/N}. \quad (16)$$

Rows correspond to y and columns to x . This small convention matters: the x multiplier must broadcast across columns, while the y multiplier must broadcast across rows.

4.2 Wavenumbers and derivatives

For even N , the FFT-ordered wavenumbers are

$$0, 1, \dots, N/2 - 1, N/2, -N/2 + 1, \dots, -1. \quad (17)$$

The code constructs these values with

```
k = np.arange(N)
k_eff = (k - N * (k > N // 2)).astype(float)
kx = (2j * np.pi * k_eff)[np.newaxis, :]
ky = (2j * np.pi * k_eff)[:, np.newaxis]
```

For a Fourier mode $e^{2\pi i k \cdot x}$, differentiation multiplies its coefficient by $2\pi i k_x$ or $2\pi i k_y$. Therefore an entire derivative is computed by one elementwise multiplication in Fourier space.

For first derivatives, the code sets the Nyquist multiplier to zero. At that frequency, the positive and negative representations coincide on an even grid, so the sign required by an odd derivative is ambiguous. This is a symmetry treatment, not a complete dealiasing method.

4.3 Inverse Laplacian and zero mode

Since

$$\Delta e^{2\pi i k \cdot x} = -4\pi^2 |k|^2 e^{2\pi i k \cdot x}, \quad (18)$$

the inverse-Laplacian multiplier is

$$\widehat{\Delta^{-1} f}_k = -\frac{\widehat{f}_k}{4\pi^2 |k|^2}, \quad k \neq 0. \quad (19)$$

The zero mode is set to zero because constants lie in the kernel of the Laplacian. This selects the mean-zero solution. In code:

```
lap = kx**2 + ky**2
LAP_INV = np.zeros_like(lap)
nz = lap != 0
LAP_INV[nz] = 1.0 / lap[nz]
```

4.4 Discrete norm factors

For grid samples on a unit square,

$$\|f\|_{L^p} \approx \left(h^2 \sum_{i,j} |f_{ij}|^p \right)^{1/p} = h^{2/p} \|f\|_{\ell^p}. \quad (20)$$

Consequently the code multiplies the vector ℓ^2 , ℓ^4 , and ℓ^8 norms by h , $h^{1/2}$, and $h^{1/4}$, respectively. For the unnormalized NumPy FFT, Parseval gives

$$\|f\|_{L^2} \approx \frac{\|\widehat{f}\|_{\ell^2}}{N^2}. \quad (21)$$

The factor N^{-2} in the velocity normalization is therefore essential: without it, the effective stirring budget would change with resolution.

5 The Right-Hand Side Algorithm

At each Runge–Kutta stage, the solver receives $\widehat{\theta}$ and must return $\partial_t \widehat{\theta}$. Algorithmically, the calculation is:

1. Compute $\phi = \Delta^{-1} \theta$ in Fourier space.
2. Transform θ , $\partial_x \phi$, and $\partial_y \phi$ to physical space.
3. Multiply pointwise to obtain $g = (\theta \partial_x \phi, \theta \partial_y \phi)$, then transform both components back to Fourier space.
4. Apply the Leray projection (9) mode by mode.
5. Apply $-\Delta^{-1}$ to obtain v .
6. Compute $\|\nabla v\|_{L^2}$ using four spectral derivatives and the Parseval factor in (21).

7. Normalize v to obtain u with $\|\nabla u\|_{L^2} = F$.
8. Transform u , $\partial_x \theta$, and $\partial_y \theta$ to physical space; form $u \cdot \nabla \theta$; transform back; negate.

The core implementation follows the same order:

```
linv_th = LAP_INV * theta_hat
theta = np.real(np.fft.ifft2(theta_hat))
gx_hat = np.fft.fft2(
    theta * np.real(np.fft.ifft2(DEL_X * linv_th)))
gy_hat = np.fft.fft2(
    theta * np.real(np.fft.ifft2(DEL_Y * linv_th)))

div_g = DEL_X * gx_hat + DEL_Y * gy_hat
pgx = gx_hat - DEL_X * LAP_INV * div_g
pgy = gy_hat - DEL_Y * LAP_INV * div_g

vx_hat = -LAP_INV * pgx
vy_hat = -LAP_INV * pgy
```

The final transport step is

```
dtheta_hat = -np.fft.fft2(
    ux * np.real(np.fft.ifft2(DEL_X * theta_hat))
    + uy * np.real(np.fft.ifft2(DEL_Y * theta_hat)))
```

5.1 Why products are formed in physical space

Multiplication in physical space is convolution in Fourier space. A direct two-dimensional convolution over all mode pairs costs $O(N^4)$. Two FFTs, a pointwise multiplication, and an inverse FFT cost $O(N^2 \log N)$, which is why the method is called pseudo-spectral.

The speed comes with aliasing. Products create frequencies above the grid's Nyquist limit, and those frequencies fold onto lower modes. The current code does not apply explicit dealiasing. A standard improvement would retain the lower two-thirds of the modes and use 3/2 zero-padding for each quadratic product. The conservation event detects accumulated resolution loss, but it does not prevent aliasing before that threshold is crossed.

6 State Representation and Adaptive Time Integration

SciPy's `solve_ivp` expects a real vector in the workflow used here, whereas $\hat{\theta}$ is complex. The code stores

$$y = \begin{bmatrix} \text{Re}(\hat{\theta}) \\ \text{Im}(\hat{\theta}) \end{bmatrix} \in \mathbb{R}^{2N^2}. \quad (22)$$

Each call to the right-hand side reconstructs the complex $N \times N$ array, performs the spectral calculation, and returns the real and imaginary parts in the same flattened order.

The time integrator is RK45, the Dormand–Prince embedded pair. At each candidate step it computes approximations of orders five and four. Their difference estimates the

local truncation error. The step is accepted when the scaled error is below one and rejected otherwise; the next step size is adjusted according to the observed error. Here

$$\text{rtol} = 10^{-6}, \quad \text{atol} = 10^{-8}. \quad (23)$$

The relative tolerance controls error proportional to the solution scale, while the absolute tolerance protects components near zero.

7 Resolution Check as an Event Function

Exact incompressible transport preserves every L^p norm. A numerical change in these norms therefore signals spatial or temporal error. The code monitors

$$E(t) = \max \left(|L_2(t) - 1|, \left| \frac{L_4(t)}{L_4(0)} - 1 \right|, \left| \frac{L_8(t)}{L_8(0)} - 1 \right| \right) - 10^{-3}. \quad (24)$$

SciPy searches for an upward zero crossing of E . When one is found, the event is terminal and integration stops.

The higher norms serve a purpose. L^8 weights extreme localized values more strongly than L^2 , so it can reveal damage to sharp filaments sooner. Using all three norms makes the diagnostic sensitive to different forms of under-resolution.

Interpretation 7.1. The event is an error detector, not an error estimator with a rigorous bound on every observable. A mixing rate fitted close to the stopping time can still depend on resolution, which is exactly what the $N = 32$ and $N = 64$ comparison demonstrates.

8 Initial Data and Normalization

Each initial-data function performs three conceptual operations:

1. evaluate an analytic pattern on the grid;
2. restrict or rearrange its support;
3. divide by its discrete L^2 norm and shift it to the domain centre.

For the sine bump,

$$\theta_0(x, y) = \sin(2\pi x/a) \sin(2\pi y/a) \mathbf{1}_{[0,a)^2}(x, y), \quad (25)$$

before normalization and centering. The parameter a changes both the support size and the characteristic spatial frequency. Smaller a begins with finer structure and reaches the grid scale sooner.

Normalization is implemented as

```
f /= np.linalg.norm(f.ravel()) * dx
```

because $dx = h$ and, in two dimensions, $\|f\|_{L^2} \approx h \|f\|_{\ell^2}$.

9 Extracting and Interpreting Mixing Rates

If the normalized mix norm obeys

$$M(t) = \frac{\|\theta(t)\|_{H^{-1}}}{\|\theta_0\|_{H^{-1}}} \approx e^{-rt}, \quad (26)$$

then

$$\log M(t) \approx -rt. \quad (27)$$

A least-squares line fitted to $\log M$ therefore has slope $-r$, and the mixing timescale is $\tau = 1/r$. The thesis fits the last two-thirds of each trajectory to reduce the influence of the initial transient.

For $a = 0.5$ and $N = 64$, the reported late-window fit gives $r \approx 0.440$ and $\tau \approx 2.27$. A full-trajectory fit gives $r \approx 0.394$. Both values are correct for their respective windows; the difference shows that the curve is only approximately log-linear.

Across eight values of a , a second log-log regression fits

$$r(a) \approx Ca^{-\alpha}. \quad (28)$$

Taking logarithms produces

$$\log r \approx \log C - \alpha \log a, \quad (29)$$

so $-\alpha$ is the slope of a linear fit to $(\log a, \log r)$. The study finds $\alpha \approx 1.61$ at $N = 32$ and $\alpha \approx 1.78$ at $N = 64$. Because the exponent changes materially with N , it is not yet numerically converged. The two resolutions cannot establish its limiting value or separate finite-resolution effects from the behavior of the greedy control.

10 End-to-End Code Map

Table 1: Relationship between mathematical tasks and Python functions.

Task	Function	Main output
Build grid and multipliers	<code>build_operators</code>	Derivatives, inverse Laplacian, mix-norm weight, and coordinate arrays
Construct θ_0	<code>idata_sin</code> , <code>idata_diag</code> , <code>idata_strip</code> , <code>idata_trigpoly</code>	Centered, L^2 -normalized physical array
Evaluate the ODE	<code>make_convection_hat</code>	Closure returning $d\hat{\theta}/dt$ in real-split form
Detect resolution loss	<code>make_res_check</code>	Terminal event based on L^2 , L^4 , and L^8 drift
Integrate one case	<code>run_simulation</code>	Times, fields, spectra, and diagnostic norms
Fit decay rates	<code>replot_norms</code>	Late-window slopes, timescales, and scaling plots

The complete execution path is:

1. call `build_operators(N)` once;
2. choose an initial-data function and a value of a ;

3. construct θ_0 , transform it, and pack the real state;
4. build the right-hand side and resolution event as closures;
5. call `solve_ivp`;
6. unpack each saved state and compute diagnostic norms;
7. repeat for the parameter sweep and fit the rates.

11 Numerical Risks and Recommended Extensions

1. **Add explicit dealiasing.** Implement 3/2 padding for every quadratic product and compare against the present event-only safeguard.
2. **Run a resolution ladder.** Use at least $N = 32, 64, 128, 256$ where feasible, and compare rates over a common physical fitting interval as well as each run’s late window.
3. **Quantify fit uncertainty.** Report standard errors or bootstrap intervals for r and α , and test sensitivity to the fitting window.
4. **Guard the degenerate normalization.** Replace a diagnostic print with a documented fallback or controlled termination when $\|\nabla v\|_{L^2}$ approaches machine precision.
5. **Test invariants directly.** Add automated tests for the discrete divergence of Pg , the achieved stirring budget, Hermitian symmetry, mean preservation, and monotone initial decrease of the mix norm.
6. **Compare control strategies.** Use finite-horizon or adjoint optimization to measure the performance gap between greedy and non-greedy stirring.

12 Summary

The method can be understood as a short chain of ideas. The H^{-1} norm measures large-scale inhomogeneity. Differentiating it converts mixing into an inner product between the velocity and $\theta \nabla \Delta^{-1} \theta$. The Leray projection keeps the part that an incompressible velocity can use. Applying $-\Delta^{-1}$ moves the problem into the enstrophy inner product, and Cauchy–Schwarz identifies the steepest admissible direction. FFT multipliers realize every differential operator, while physical-space products evaluate the nonlinearity efficiently. Adaptive RK45 advances the resulting ODE, and conservation drift determines when the grid is no longer trustworthy.

This chain also clarifies the main limitation of the reported scaling law: the fitted exponent changes with resolution because the usable trajectory length changes with the grid. The present calculations establish a reproducible framework and a clear finite-resolution result, but not an asymptotic exponent.

References

- [1] Z. Lin, J.-L. Thiffeault, and C. R. Doering, Optimal stirring strategies for passive scalar mixing, *Journal of Fluid Mechanics*, 675:465–476, 2011.
- [2] G. Iyer, A. Kiselev, and X. Xu, Lower bounds on the mix norm of passive scalars advected by incompressible enstrophy-constrained flows, *Nonlinearity*, 27(5):973–985, 2014.
- [3] L. N. Trefethen, *Spectral Methods in MATLAB*, SIAM, 2000.